UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Lab Report – 06

COURSE CODE: CSE 402

COURSE TITLE: Operating System Lab

SUBMITTED BY:

NAME : Swagoto Utsab Singha Roy

STUDENT ID : 23101124

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C–1

DATE : 12 – 10 – 2026

SUBMITTED TO:

ATIA RAHMAN ORTHI
LECTURER
DEPARTMENT OF CSE, UAP

1. Problem Statement

Implement two classic **Disk Scheduling** algorithms – **FCFS** (First Come First Serve) and **SSTF** (Shortest Seek Time First) – for the following input, and compare their total seek distances.

Parameter	**Value**
Initial Head Position	20
Request Queue (as given)	0, 30, 41, 62, 14, 20
Queue after removing head	0, 30, 41, 62, 14

For each algorithm, record:

- The sequence of disk head movements (From → To → Seek Distance)
- Total Seek Distance
- Seek Sequence

Compare the two algorithms to determine which minimises total seek distance.

FCFS – First Come First Serve: Serves disk requests strictly in the order they appear in the request queue, regardless of the current head position. Simple and starvation-free, but ignores proximity, often causing large unnecessary head movements.

SSTF – Shortest Seek Time First: At each step, selects the pending request *closest* to the current head position. This greedy approach significantly reduces seek distance compared to FCFS but can cause starvation for requests far from the current head when closer ones keep arriving.

2. Source Code

```python
req  = [0, 30, 41, 62, 14, 20]
head = 20

req = [r for r in req if r != head]

print("Initial Head Position:", head)
print("Request Queue:", req)

# FCFS
total_fcfs = 0
cur = head
fcfs_seq = [head]

print("\nFCFS Total Seek Movements")
print("From\tTo\tSeek")
for r in req:
    seek = abs(r - cur)
    print(f"{cur}\t{r}\t{seek}")
    total_fcfs += seek
    cur = r
    fcfs_seq.append(r)

print("FCFS Total Seek =", total_fcfs)
print("Seek Sequence:", " -> ".join(map(str, fcfs_seq)))

# SSTF
remaining = req.copy()
total_sstf = 0
cur = head
sstf_seq = [head]

print("\nSSTF Total Seek Movements")
print("From\tTo\tSeek")
while remaining:
    closest = min(remaining, key=lambda x: abs(x - cur))
    seek = abs(closest - cur)
    print(f"{cur}\t{closest}\t{seek}")
```

```python
    total_sstf += seek
    cur = closest
    sstf_seq.append(closest)
    remaining.remove(closest)

print("SSTF Total Seek =", total_sstf)
print("Seek Sequence:", " -> ".join(map(str, sstf_seq)))

# Comparison
print("\nComparison")
print("FCFS Total Seek =", total_fcfs)
print("SSTF Total Seek =", total_sstf)

if total_sstf < total_fcfs:
    print("SSTF is better less seek time")
elif total_fcfs < total_sstf:
    print("FCFS is better less seek time")
else:
    print("Both have equal seek time")
```

*Listing 1: FCFS and SSTF Disk Scheduling (**FCFS_SSTF.py**)*

3. Output

```
Request Queue: [0, 30, 41, 62, 14]

FCFS Total Seek Movements
From    To      Seek
------------------------
20      0       20
0       30      30
30      41      11
41      62      21
62      14      48
------------------------
FCFS Total Seek  = 130
Seek Sequence: 20 -> 0 -> 30 -> 41 -> 62 -> 14

SSTF Total Seek Movements
From    To      Seek
------------------------
20      14      6
14      0       14
0       30      30
30      41      11
41      62      21
------------------------
SSTF Total Seek  = 82
Seek Sequence: 20 -> 14 -> 0 -> 30 -> 41 -> 62

Comparison
FCFS Total Seek      = 130
SSTF Total Seek      = 82
SSTF is better -- less seek time
```

Output of `FCFS_SSTF.py`

FCFS – Seek Movements:

From	To	Seek Distance
20	0	20
0	30	30
30	41	11
41	62	21
62	14	48
FCFS Total Seek		**130**

Seek Sequence: $20 \rightarrow 0 \rightarrow 30 \rightarrow 41 \rightarrow 62 \rightarrow 14$

SSTF – Seek Movements:

From	To	Seek Distance
20	14	6
14	0	14
0	30	30
30	41	11
41	62	21
SSTF Total Seek		**82**

Seek Sequence: $20 \rightarrow 14 \rightarrow 0 \rightarrow 30 \rightarrow 41 \rightarrow 62$

Comparison:

Algorithm	Total Seek Distance
FCFS	130
SSTF	**82**

SSTF is better – less seek time.

4. Discussion

In this experiment, two classic disk scheduling algorithms were implemented and compared for the given request queue starting at head position 20 (with the head's own position removed from the queue before processing).

FCFS Analysis: The head processes requests strictly in arrival order: 0, 30, 41, 62, 14. Starting at 20, it moves immediately to 0 (seek = 20), then sweeps upward to 30, 41, and 62. The final movement from 62 back to 14 (seek = 48) is particularly costly – the head traverses nearly the full disk width after having just reached the far end. This is a direct consequence of FCFS ignoring head position; it results in a total seek distance of 130.

SSTF Analysis: From position 20, the closest pending request is 14 (seek = 6). From 14, the closest is 0 (seek = 14). The algorithm then sweeps upward naturally: $0 \rightarrow 30 \rightarrow 41 \rightarrow 62$, each step choosing the nearest remaining. This greedy strategy avoids the costly backward jump that FCFS made from 62 to 14, reducing the total seek distance to 82 – a saving of 48 (37% improvement over FCFS).

Key Observations:

- SSTF outperforms FCFS here because the request distribution (clustered between 0 and 62) allows the nearest-neighbour strategy to naturally sweep in one direction without large back-and-forth movements.
- SSTF can cause starvation if requests near 30 kept arriving, the request at 62 might be postponed indefinitely.
- FCFS, despite its inefficiency, guarantees every request is served in arrival order with no risk of starvation.

5. Conclusion

In this experiment, FCFS and SSTF disk scheduling algorithms were implemented and compared. For the given request queue with initial head position 20, SSTF achieved a total seek distance of 82 compared to FCFS's 130 a 37% reduction – by always serving the nearest pending request. However, SSTF's greedy nature introduces the risk of starvation for distant requests, whereas FCFS guarantees every request is served in arrival order at the cost of higher total seek distance. These results confirm that purely greedy algorithms can be highly efficient for static workloads but may be unfair under dynamic, continuous request streams.

6. GitHub

Click here to view source code